Apunte teórico - Lab. Algoritmos y Estructura de Datos (parte 3)

ÍNDICE

1. Funciones con retorno
2. Arreglos/Vectores como parámetros
3. ¿Qué es un String en C?
4. EL '\0'
5. Libreria <string.h>
6. Recorrer un String
7. Longitud de un String
8. Comparar Strings
9. Buscar caracteres dentro de un String
10. Buscar subcadenas
11. Modificar Strings
12. Convertir mayúsculas a minúsculas
13. Invertir un String
14. Palíndromos
15. Punteros
16. Direcciones de memoria
17. Operador &
18. Operador *
19. & y * la diferencia fundamental
20. Paso por referencia
21. Modificar una variable desde una función
22. Punteros y funciones
23. Devolver varios valores mediante punteros
24. Ejemplo: Max y Min
25. Punteros y arreglos
26. Structs
27. Acceder a los campos de un Struct
28. Structs y funciones
29. Struct por valor
30. Punteros a Structs
31. Operador ->
32. Arreglos de Structs
33. Recorrer un arreglo de Structs
34. Funciones con arreglos de Structs
35. Structs con arreglos internos
36. Structs anidados

37. Combinar Structs + Punteros

38. Arreglos de Structs + Punteros

39. Ejemplo completo: Transferencia Bancaria

40. Resumen de simbolos

---

1. FUNCIONES CON RETORNO

Ya conocemos qué es una función.

Una función es un bloque de código que realiza una tarea específica.

Por ejemplo:

```
int doble(int numero)
{
   return numero * 2;
}
```

Podemos pensar una función como una máquina:

Entrada → proceso → resultado

En este caso:

5 → función doble → 10

¿Qué significa return?

return permite que una función **devuelva un resultado**.

Ejemplo:

```
int sumar(int a, int b)
{
   return a + b;
}
```

Si hacemos:

```
int resultado = sumar(5, 3);
```

La función hace:

5 + 3

↓

8

Y devuelve:

8

Entonces:

resultado = 8;

Ejemplo

```
float calcularDescuento(float precio, float descuento)
{
   return precio - (precio * descuento / 100);
```

}
Podemos hacer:
float precioFinal = calcularDescuento(1000, 20);
Resultado:
800

---

2. ARREGLOS/VECTORES COMO PARÁMETROS
Una función puede recibir un **vector/arreglo** como parámetro.
Por ejemplo:

```
void mostrarVector(int vector[], int tamano)
{
   for(int i = 0; i < tamano; i++)
   {
      printf("%d\n", vector[i]);
   }
}
```

Tenemos:
void mostrarVector(int vector[], int tamano)
Dos parámetros:
vector → arreglo que queremos recorrer
tamano → cantidad de elementos

¿Por qué tenemos que enviar también el tamaño?
Porque C necesita saber hasta dónde recorrer el arreglo.
Por ejemplo:
int numeros[5] = {10, 20, 30, 40, 50};
La función puede recibir:
mostrarVector(numeros, 5);
Entonces sabe que debe recorrer:
posición 0
posición 1
posición 2
posición 3
posición 4

Ejemplo: sumar elementos

```
int sumarVector(int vector[], int tamano)
{
   int suma = 0;
```

```
    for(int i = 0; i < tamano; i++)
    {
        suma += vector[i];
    }

    return suma;
}
```

Uso:

```
int numeros[4] = {10, 20, 30, 40};

int resultado = sumarVector(numeros, 4);
```

Resultado:

100

Algo **MUY** importante

Cuando pasamos un arreglo a una función:

```
sumarVector(numeros, 4);
```

No hacemos:

```
sumarVector(numeros[0], 4);
```

Porque numeros[0] es **un solo elemento**.

En cambio:

numeros

representa el arreglo que estamos enviando.

---

3. ¿QUÉ ES UN STRING EN C?

En C **no existe un tipo de dato llamado string**.

Para guardar texto utilizamos un:

char[]

Es decir: **un String es un arreglo de caracteres.**

Ejemplo

```
char nombre[] = "Agustin";
```

Esto significa que tenemos un arreglo de caracteres:

A

g

u

s

t

i

n

Cada posición contiene un char.

Podemos imaginarlo:
nombre

[A][g][u][s][t][i][n][\0]
Hay algo MUY importante al final: **'\0'**

---

4. EL '\0'
El '\0' indica: **Acá termina el String.**
Esto es fundamental.
Por ejemplo:
char nombre[] = "Hola";
Internamente tenemos:
[H][o][l][a][\0]
Por eso un String necesita una posición adicional para guardar el '\0'.
¿Por qué existe?
Porque C necesita alguna manera de saber: "¿Hasta dónde llega este texto?"
Entonces recorre caracteres hasta encontrar: '\0'
char vs String
Esto es importantísimo:
char letra = 'A';
Guarda **un solo carácter**.
En cambio:
char nombre[] = "Agustin";
guarda **una cadena de caracteres**.
Un carácter:
'A'
Un String:
"Agustin"
Ojo con las comillas:
'A' → comilla simple → carácter.
"Agustin" → comilla doble → String.
IMPORTANTE: si queremos imprimir un string en C debemos usar %s (que viene de string), como cuando antes usabamos %d para imprimir enteros, o %f para imprimir flotantes. Aqui usaremos %s.
char nombre[] = "alumno";
printf("String: %s", nombre); → imprime “alumno”

---

5. LIBRERIA <STRING.H>

C ya tiene una librería que nos proporciona funciones para trabajar con Strings:

#include <string.h>

Esta librería contiene funciones preparadas para realizar operaciones comunes sobre cadenas.

Las más importantes para estos ejercicios son:

Función	¿Qué hace?
strlen()	Obtiene la longitud
strcpy()	Copia un String
strncpy()	Copia una cantidad determinada de caracteres
strcmp()	Compara dos Strings
strncmp()	Compara una cantidad determinada de caracteres
strcat()	Une dos Strings
strncat()	Une una cantidad determinada de caracteres
strchr()	Busca un carácter
strstr()	Busca un substring

---

1. strlen() — obtener la longitud

strlen viene de: string length

Es decir: longitud del String.

Se utiliza: strlen(cadena)

Por ejemplo:

#include <string.h>

char cadena[] = "Hola";

int longitud = strlen(cadena);

→ longitud: 4

¿Qué cuenta?

Cuenta los caracteres hasta encontrar:

'\0'

Por eso:

H o l a \0

0 1 2 3 4

La longitud es:

4
El '\0' no se cuenta.

---

2. strcpy() — copiar un String
strcpy significa: string copy
Sirve para copiar una cadena dentro de otra.
Por ejemplo:
char origen[] = "Hola";
char destino[20];
strcpy(destino, origen);
Ahora:
origen → Hola
destino → Hola
La sintaxis es:
strcpy(destino, origen);
El destino va primero.

---

3. strcmp() — comparar Strings
Esta es muy importante.
Si queremos saber si dos Strings son iguales, no hacemos:
if(cadena1 == cadena2)
Eso no compara correctamente el contenido de las cadenas.
Para comparar Strings usamos:
strcmp(cadena1, cadena2)
Por ejemplo:

```
if(strcmp(cadena1, cadena2) == 0)
{
   printf("Son iguales");
}
```

¿Qué devuelve strcmp()?
Devuelve:
0 → los Strings son iguales
<0 → el primero es menor
>0 → el primero es mayor
Para el caso más importante:
strcmp("Hola", "Hola")
devuelve:
0
Por lo tanto: son iguales

---

4. strcat() — unir Strings
strcat significa: string concatenate
Sirve para agregar una cadena al final de otra.
Por ejemplo:
char nombre[30] = "Juan";
char apellido[] = "Perez";

strcat(nombre, apellido);
El resultado sería:
JuanPerez
Si queremos un espacio:
char nombre[30] = "Juan ";
char apellido[] = "Perez";

strcat(nombre, apellido);
Resultado: Juan Perez

---

5. strchr() — buscar un carácter
strchr sirve para: buscar un carácter dentro de un String.
Por ejemplo:
char cadena[] = "banana";
strchr(cadena, 'a');
Busca:
'a'
dentro de:
banana
La función devuelve un puntero a la primera aparición encontrada.
Por ejemplo:
char cadena[] = "banana";
char *resultado = strchr(cadena, 'a');
Si encontramos 'a', resultado apunta a esa posición.

---

6. strstr() — buscar un substring
strstr sirve para buscar un String dentro de otro String.
Por ejemplo:
Cadena:
"programacion"
Buscar:
"gram"
Podemos hacer:

strstr(cadena, "gram");

Si lo encuentra, devuelve un puntero a donde comienza ese substring.

Esto puede simplificar muchísimo el ejercicio que pide comprobar si una cadena contiene una subcadena, que aparece en la lista de ejercicios de Strings.

---

7. strstr() vs comprobar si empieza con una subcadena

Hay una diferencia importante.

Si tenemos:

"programacion"

y buscamos:

"gram"

strstr() lo encuentra:

proGRAMacion

↑

Pero si el ejercicio pregunta: ¿La cadena comienza con esta subcadena?

Entonces:

"programacion"

"pro"

→ Sí.

Pero:

"programacion"

"gram"

→ No.

Aunque strstr() encuentre "gram".

Por eso el ejercicio de "comienza con" puede resolverse recorriendo las primeras posiciones, como vimos anteriormente.

---

8. strncpy() — copiar una cantidad de caracteres

Es una versión de strcpy() que permite indicar cuántos caracteres copiar.

strncpy(destino, origen, cantidad);

Por ejemplo:

char origen[] = "Programacion";

char destino[20];

strncpy(destino, origen, 4);

Copia:

Prog

Hay una cuestión importante: strncpy() no siempre agrega automáticamente '\0' si se copian exactamente los primeros n caracteres de una cadena suficientemente larga.

Por eso, para principiantes, conviene tener presente que puede ser necesario hacer:

destino[4] = '\0';

---

9. strncmp() — comparar una cantidad de caracteres

Es parecido a:

strcmp()

pero podemos decir cuántos caracteres comparar.

strncmp(cadena1, cadena2, cantidad);

Por ejemplo:

strncmp("Hola", "Hola mundo", 4)

compara solamente:

Hola

Hola

y devuelve:

0

porque los primeros 4 caracteres son iguales.

---

10. strncat() — concatenar una cantidad determinada

Es parecido a:

strcat()

pero permite especificar cuántos caracteres del segundo String agregar.

strncat(destino, origen, cantidad);

Por ejemplo:

```
char cadena[30] = "Hola ";
char otra[] = "mundo123";
strncat(cadena, otra, 5);
```

Resultado:

Hola mundo

porque agrega solamente:

mundo

---

6. RECORRER UN STRING

Como un String es un arreglo de char, podemos recorrerlo con un for.

Ejemplo:

```
char nombre[] = "Hola";

for(int i = 0; nombre[i] != '\0'; i++)
{
    printf("%c\n", nombre[i]);
}
```

Resultado:

H

o
l
a

¿Qué está haciendo el for?
La condición: nombre[i] != '\0'
Significa: Mientras el carácter actual NO sea el final del String, seguimos.
Entonces:
i = 0 → H
i = 1 → o
i = 2 → l
i = 3 → a
i = 4 → \0 → termina

Esta forma es MUY importante
Cuando trabajemos con Strings, vamos a ver muchísimo:
for(int i = 0; cadena[i] != '\0'; i++)
Aprendétela conceptualmente: **"Recorro el String hasta encontrar el final."**

---

7. LONGITUD DE UN STRING
Muchas veces necesitamos saber cuántos caracteres tiene un String.
Podemos hacerlo manualmente:

```
int longitud(char cadena[])
{
   int contador = 0;

   while(cadena[contador] != '\0')
   {
      contador++;
   }

   return contador;
}
```

Si tenemos:
char palabra[] = "Hola";
La función devuelve: **4**
¿Por qué devuelve 4 y no 5?
Porque:
H → 1
o → 2

l → 3
a → 4
\0 → no es parte del texto
El '\0' sirve para marcar el final, pero **no lo contamos como carácter del String**.

---

8. COMPARAR STRINGS
Acá hay una diferencia MUY importante respecto a los números.
Con números podemos hacer:
if(a == b)
Pero con Strings **no debemos comparar directamente**:
if(nombre1 == nombre2)
Porque los Strings son arreglos.
Para comparar su contenido podemos **recorrerlos carácter por carácter.**
Comparación manual
Supongamos:
char palabra1[] = "hola";
char palabra2[] = "hola";
Queremos saber si son iguales.
Conceptualmente debemos comprobar:
posición 0 → h == h
posición 1 → o == o
posición 2 → l == l
posición 3 → a == a
posición 4 → \0 == \0
Podemos hacer una función:

```
bool sonIguales(char a[], char b[])
{
   int i = 0;

   while(a[i] != '\0' && b[i] != '\0')
   {
      if(a[i] != b[i])
      {
         return false;
      }

      i++;
   }

   return a[i] == b[i];
```

}

Básicamente el bucle itera siempre que el arreglo de caracteres (a y b) sea distinto de su carácter final \0. En caso de que la posición en a que este el bucle, el carácter del arreglo a sea distinto al de b, inmediatamente la funcion devuelve false, caso contrario itera la i hasta finalizar y termina retornando el arreglo.

---

9. BUSCAR CARACTERES DENTRO DE UN STRING

Pedir una cadena y un carácter y mostrar cuántas veces aparece ese carácter.

Por ejemplo:

Cadena: "banana"

Carácter: 'a'

Tenemos:

b a n a n a

↑ ↑ ↑

Aparece: 3 veces

Función

```
int contarCaracter(char cadena[], char caracter)
{
   int contador = 0;

   for(int i = 0; cadena[i] != '\0'; i++)
   {
      if(cadena[i] == caracter)
      {
         contador++;
      }
   }

   return contador;
}
```

Uso:

```
int cantidad = contarCaracter("banana", 'a');
```

Resultado: 3

¿Qué estamos combinando? Acá ya estamos mezclando:

- Strings
- arreglos
- char
- for
- if

- comparación
- contador
- funciones

Por eso estos ejercicios empiezan a ser bastante más completos. Lo fundamental es comprender las bases vistas en los apuntes anteriores.

---

10. BUSCAR SUBCADENAS

Una **subcadena** es una cadena que aparece dentro de otra.

Ejemplo:

Cadena: "Hola mundo"

Subcadena: "mundo"

Sí está contenida.

Otro ejemplo:

Cadena: "programacion"

Subcadena: "gram"

También está contenida.

Uno de los ejercicios pide comprobar si una cadena comienza con una subcadena.

Por ejemplo:

Cadena: "programacion"

Subcadena: "pro"

Resultado: Sí

Pero:

Cadena: "programacion"

Subcadena: "gram"

Resultado: No

Porque no **comienza** con "gram".

Idea para resolverlo

Comparamos los caracteres iniciales.

programacion

pro

Comparar:

p == p

r == r

o == o

Todos coinciden.

Entonces comienza con esa subcadena.

El ejemplo seria algo asi:

bool comienzaCon(char cadena[], char subcadena[])

```
{
   int i = 0;
   while(subcadena[i] != '\0')
   {
      if(cadena[i] != subcadena[i])
      {
         return false;
      }
      i++;
   }
   return true;
}
```

---

11. MODIFICAR STRINGS

Como un String es un arreglo de char, podemos modificar sus posiciones.

Por ejemplo:

```
char palabra[] = "hola";
```

Podemos hacer:

```
palabra[0] = 'H';
```

Ahora:

Hola

Sustituir un carácter

Uno de los ejercicios pide: Sustituir la aparición de un carácter por otro.

Por ejemplo:

Cadena: "banana"

Buscar: 'a'

Reemplazar: 'o'

Resultado: bonono

Podemos recorrer el String:

```
for(int i = 0; cadena[i] != '\0'; i++)
{
   if(cadena[i] == buscar)
   {
      cadena[i] = reemplazar;
   }
}
```

La idea es:

¿Este carácter es el que estoy buscando?

SI → lo reemplazo
NO → sigo
Quedaria algo asi:

```
#include <stdio.h>

int main()
{
    char cadena[] = "banana";
    char buscar = 'a';
    char reemplazar = 'o';

    for(int i = 0; cadena[i] != '\0'; i++)
    {
        if(cadena[i] == buscar)
        {
            cadena[i] = reemplazar;
        }
    }
    printf("Resultado: %s", cadena);
    return 0;
}
```

---

12. CONVERTIR MAYÚSCULAS Y MINÚSCULAS

Otro ejercicio pide: Convertir mayúsculas a minúsculas y minúsculas a mayúsculas.
Por ejemplo:
Hola Mundo
se transforma en:
hOLA mUNDO
La idea general es recorrer:

```
for(int i = 0; cadena[i] != '\0'; i++)
{
    ...
}
```

Y analizar cada carácter.

¿Cómo sabemos si es mayúscula?
En C, los caracteres tienen valores asociados.
Por ejemplo:
'A'
'B'

'C'
...
'Z'
forman un rango.
Podemos comprobar:
if(cadena[i] >= 'A' && cadena[i] <= 'Z')
Entonces sabemos: El carácter está entre A y Z → es mayúscula.
Y para minúsculas:
if(cadena[i] >= 'a' && cadena[i] <= 'z')

```
void cambiarMayusculasMinusculas(char cadena[])
{
   for(int i = 0; cadena[i] != '\0'; i++)
   {
      if(cadena[i] >= 'A' && cadena[i] <= 'Z')
      {
         cadena[i] = cadena[i] + 32;
      }
      else if(cadena[i] >= 'a' && cadena[i] <= 'z')
      {
         cadena[i] = cadena[i] - 32;
      }
   }
}
```

La diferencia entre dos letras minuscula y mayuscula siempre es de 32 en la tabla ASCII

---

13. INVERTIR UN STRING
Dada una cadena, generar otra cadena resultado de invertir la primera.
Por ejemplo:
Original:
Hola
Invertida:
aloH
Idea
Si tenemos:
H o l a
0 1 2 3
Para invertirlo debemos acceder desde el final:
a l o H ←
Entonces podemos pensarlo como:

```
void invertirString(char cadena[], char invertida[], int longitud)
{
   for(int i = 0; i < longitud; i++)
   {
      invertida[i] = cadena[longitud - 1 - i];
   }

   invertida[longitud] = '\0';
}
```

La idea que quiero es esta:

cadena:

H O L A

0 1 2 3

↓

↓ invertimos

↓

invertida:

A L O H \0

0 1 2 3 4

Y longitud - 1 sirve para obtener las posiciones 3, 2, 1, 0, es decir, recorrer la cadena original de atrás hacia adelante.

---

14. PALÍNDROMOS

Una palabra palíndroma se lee igual de adelante hacia atrás.

Ejemplos:

radar

reconocer

neuquen

Por ejemplo:

radar

Desde adelante:

r a d a r

Desde atrás:

r a d a r

Son iguales.

Si encontramos una diferencia:

No es palíndromo

```
bool esPalindromo(char cadena[], int longitud)
{
   for(int i = 0; i < longitud / 2; i++)
   {
      if(cadena[i] != cadena[longitud - 1 - i])
      {
         return false;
      }
   }

   return true;
}
```

¿Cómo funciona?

Supongamos: radar

Tenemos:

r a d a r

0 1 2 3 4

Comparamos los extremos:

cadena[0] con cadena[4]

r r

Son iguales.

Después:

cadena[1] con cadena[3]

a a

Son iguales.

No hace falta comparar el centro porque ya llegamos a la mitad.

Por eso usamos:

i < longitud / 2

¿Y si encontramos una diferencia?

Por ejemplo: hola

h o l a

0 1 2 3

Primero:

'h' != 'a'

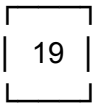
Entonces:

return false;

La función termina inmediatamente y sabemos que no es palíndromo.
Si recorremos todas las comparaciones sin encontrar diferencias:
return true;
Entonces sí es palíndromo.

---

15. PUNTEROS
Ahora viene probablemente **uno de los temas más importantes y más difíciles** de todo el material.
Un puntero es una variable que **guarda una dirección de memoria**.
Hasta ahora teníamos:
int edad = 19;
Podemos imaginar:
edad



Pero esa variable está guardada en algún lugar de la memoria.
Por ejemplo, imaginariamente:
Dirección: 1000
Valor: 19
Entonces:
edad
↓
1000 → 19
Un puntero nos permite guardar: 1000

---

16. DIRECCIONES DE MEMORIA
Cada variable está almacenada en una posición de memoria.
Podemos obtener su dirección utilizando: &
Por ejemplo:
int edad = 19;
printf("%p", &edad);

%p le indica a printf que lo que vas a imprimir es una dirección de memoria (un puntero).
int edad = 19;
printf("%d", edad); // 19
printf("%p", &edad); // dirección de memoria
La diferencia está en qué le estás pasando a printf:

- edad → contiene el valor 19 → usamos %d porque es un int.

- &edad → contiene la dirección de memoria donde está guardado edad → usamos %p.

Por ejemplo, podrías obtener algo como:

19

0x7ffe1234abcd

Ese 0x7ffe1234abcd no es el contenido de edad, sino la ubicación de edad en la memoria RAM.

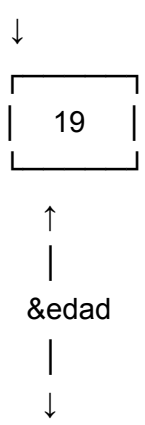
¿Y por qué no %d?

Porque %d espera recibir un int:

printf("%d", edad);

Mientras que %p está diseñado específicamente para punteros/direcciones

Pero para entender el concepto, quedate con esta idea:



"¿Dónde está esta cajita?"

Entonces:

edad → ¿QUÉ hay? → 19

&edad → ¿DÓNDE está? → 0x7ffe...

Y por eso %p = pointer (puntero).

&edad significa: **La dirección de memoria de edad.**

---

17. OPERADOR &

El operador: &

Significa: "Dame la dirección de memoria de esta variable."

Ejemplo:

int edad = 19;

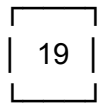
Tenemos:

&edad

Eso nos da la dirección donde está guardada edad.

Ejemplo conceptual

edad



dirección 1000

Entonces:

&edad → 1000

---

18. OPERADOR *

El * en punteros tiene otro significado.

Si tenemos:

int *puntero;

Significa: puntero es un puntero a un int.

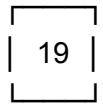
Podemos hacer:

int edad = 19;

int *puntero = &edad;

Ahora:

edad



puntero

El puntero guarda la dirección de edad.

¿Qué significa *puntero?

Significa: "Dame el valor que está guardado en la dirección a la que apunta el puntero."

Si:

puntero → edad

entonces:

*puntero

nos da: 19

---

19. & Y * – LA DIFERENCIA FUNDAMENTAL

Tenemos:

int edad = 19;

int *puntero = &edad;
Entonces:
&edad
Significa: ¿Dónde está edad?
Devuelve su **dirección**.
puntero
Contiene: La dirección de edad.
*puntero
Significa: ¿Qué valor hay en esa dirección?
Devuelve: 19

Una analogía
Imaginemos una casa.
Casa = variable
Dirección de la casa = &variable
Papel donde anoté la dirección = puntero
Entrar a la casa y ver qué hay = *puntero (valor que almacena la variable)
Entonces:
& → dirección
* → valor al que apunta

En resumen la diferencia seria:
& → obtener la **DIRECCIÓN** de memoria
puntero → una variable que **GUARDA** una dirección de memoria
***** → ir a esa dirección y "**AGARRAR**" el VALOR que hay ahí

---

20. PASO POR REFERENCIA
Hasta ahora, cuando pasábamos una variable a una función:
void cambiar(int numero)
la función recibe una copia del valor.
Por ejemplo:

```
void cambiar(int numero)
{
   numero = 100;
}
```

Si hacemos:

```
int x = 5;
cambiar(x);
```

x sigue valiendo: 5
¿Por qué? Porque la función recibió una **copia**.

---

21. MODIFICAR UNA VARIABLE DESDE UNA FUNCIÓN

Con punteros podemos hacer que la función modifique la **variable original**.

Ejemplo:

```
void cambiar(int *numero)
{
    *numero = 100;
}
```

Y en main:

```
int x = 5;
cambiar(&x);
```

Ahora: x = 100

¿Qué ocurrió?

Mandamos: &x

Es decir: La dirección de x.

La función recibe:

int *numero

Es decir: Un puntero que apunta a un int.

Y: *numero = 100;

Significa: Guardar 100 en la variable ubicada en esa dirección.

Como esa dirección era la de x:

x = 100

---

22. PUNTEROS Y FUNCIONES

Por ejemplo:

```
void incrementar(int *numero, int n)
{
    *numero += n;
}
```

Uso:

```
int x = 10;

incrementar(&x, 5);
```

Resultado:

x = 15

¿Por qué hacemos &x?

Porque la función necesita saber: "¿Qué variable tengo que modificar?"

Le damos su dirección: &x

---

23. DEVOLVER VARIOS VALORES MEDIANTE PUNTEROS

Normalmente una función puede devolver **un solo resultado** mediante return.
Por ejemplo:

```
int sumar(int a, int b)
{
   return a + b;
}
```

Pero podemos necesitar devolver varios resultados.
Por ejemplo: Quiero calcular cociente y resto.
Podemos hacer:

```
void dividir(int dividendo, int divisor, int *cociente, int *resto)
{
   *cociente = dividendo / divisor;
   *resto = dividendo % divisor;
}
```

Uso:

```
int cociente;
int resto;

dividir(10, 3, &cociente, &resto);
```

Resultado:
cociente = 3
resto = 1
¿Por qué funciona?
Mandamos las direcciones:
&cociente
&resto
La función puede modificar directamente esas variables.

---

24. EJEMPLO: MIN Y MAX
Recibir un arreglo y devolver mediante punteros el mínimo y máximo.
Podemos hacer:

```
void minMax(int vector[], int tamano, int *minimo, int *maximo)
{
   *minimo = vector[0];
   *maximo = vector[0];

   for(int i = 1; i < tamano; i++)
   {
      if(vector[i] < *minimo)
      {
```

```
            *minimo = vector[i];
        }

        if(vector[i] > *maximo)
        {
            *maximo = vector[i];
        }
    }
}
```

Uso:

```
int minimo;
int maximo;

minMax(numeros, 5, &minimo, &maximo);
```

---

25. PUNTEROS Y ARREGLOS

Esto es importante porque los vectores y los punteros están muy relacionados.

Tenemos:

```
int numeros[3] = {10, 20, 30};
```

Podemos acceder:

numeros[0], numeros[1], numeros[2]

También existe una relación con las direcciones de memoria.

Conceptualmente:

```
numeros
  ↓
[10][20][30]
  ↑
primera posición
```

Los arreglos y los punteros están muy relacionados porque el **nombre** de un arreglo representa la **dirección** de su **primer elemento**. Por eso podemos usar punteros para recorrer y modificar arreglos, y podemos pasar arreglos a funciones para que trabajen sobre el arreglo original.

```
int numeros[3] = {10, 20, 30};
int *p = numeros;
printf("%d", *p);       // 10
printf("%d", *(p + 1)); // 20
printf("%d", *(p + 2)); // 30
```

Por eso podemos trabajar con arreglos y punteros.

---

26. STRUCTS

Una struct permite **agrupar varios datos relacionados bajo un mismo nombre**.
Por ejemplo, queremos representar una persona.
Una persona puede tener:

- nombre
- edad
- altura

En vez de tener:
char nombre[50];
int edad;
float altura;
por separado, podemos agruparlos.

Crear una Struct

```
struct Persona
{
   char nombre[50];
   int edad;
   float altura;
};
```

Ahora creamos una variable:
struct Persona persona1;
Podemos pensarlo como un “molde” para crear siempre con la misma “forma” distintas cosas.

---

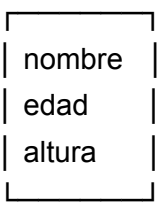
27. ACCEDER A LOS CAMPOS DE UN STRUCT
Utilizamos: **.**
Ejemplo:

```
persona1.edad = 20;
persona1.altura = 1.75;
```

Para leer:

```
printf("%d", persona1.edad);
```

Pensalo como una caja con cajitas adentro
Tenemos:
persona1



Entonces:

persona1.edad

Significa: El campo edad de persona1.

Este concepto está muy relacionado a Base de datos y POO (Programación Orientada a Objetos)

---

28. STRUCTS Y FUNCIONES

Podemos enviar una Struct a una función.

Por ejemplo, tenemos:

```
struct Punto
{
    int x;
    int y;
};
```

Podemos hacer:

```
void imprimirPunto(struct Punto p)
{
    printf("(%d, %d)", p.x, p.y);
}
```

Uso:

```
struct Punto punto;

punto.x = 10;
punto.y = 20;

imprimirPunto(punto);
```

Resultado:

(10, 20)

Ahora vamos con un ejemplo un poco mas interesante: modelar a un alumno que tiene 3 notas y queremos saber su promedio.

```
#include <stdio.h>

struct Alumno
{
    char nombre[30];
    float nota1;
    float nota2;
    float nota3;
};
```

```
float calcularPromedio(struct Alumno alumno)
{
    return (alumno.nota1 + alumno.nota2 + alumno.nota3) / 3;
}

int main()
{
    struct Alumno alumno;

    alumno.nota1 = 8;
    alumno.nota2 = 6;
    alumno.nota3 = 9;

    float promedio = calcularPromedio(alumno);

    printf("Promedio: %.2f", promedio);

    return 0;
}
```

---

29. STRUCT POR VALOR

Cuando hacemos:

```
void imprimirPunto(struct Punto p)
```

estamos pasando la Struct **por valor**.

La función recibe una copia.

Esto está bien si solamente queremos:

- leer los datos
- calcular algo
- mostrar algo

Por ejemplo:

```
float calcularDistancia(struct Punto p1, struct Punto p2)
{
    ...
}
```

Cuando pasamos una struct por valor, la función recibe una copia de la estructura original.

Por eso podemos leer sus datos y hacer cálculos con ellos, pero si modificamos la estructura dentro de la función, **el original no cambia**.

---

30. PUNTEROS A STRUCTS

Ahora podemos combinar:

Struct + puntero

Por ejemplo:

```
struct Empleado
{
   char nombre[50];
   int edad;
   float salario;
};
```

Podemos crear:

```
struct Empleado empleado;
```

Y pasar su dirección:

```
aumentarSalario(&empleado, 10);
```

La función:

```
void aumentarSalario(struct Empleado *emp, float porcentaje)
{
   emp->salario += emp->salario * porcentaje / 100;
}
```

---

31. OPERADOR ->

Acá aparece:

->

Es importantísimo.

Cuando tenemos una Struct normal:

empleado.salario

usamos:

.

Pero cuando tenemos un **puntero a una Struct**:

struct Empleado *emp;

usamos:

->

Entonces:

emp->salario

Significa: Acceder al campo salario de la Struct a la que apunta emp.

Comparación

Struct normal

empleado.salario

Puntero a Struct

emp->salario

Una forma de entenderlo
Si:
struct Empleado *emp;
entonces:
emp->salario
es equivalente conceptualmente a:
(*emp).salario
Primero:
*emp
→ obtenemos la Struct apuntada.
Después:
.salario
→ accedemos al campo.
Por eso:
emp->salario
es simplemente una forma más cómoda de escribirlo.

---

32. ARREGLOS DE STRUCTS
Podemos tener un vector donde cada posición sea una Struct.
Por ejemplo:

```
struct Alumno
{
   char nombre[50];
   int legajo;
   float promedio;
};
```

Creamos:
struct Alumno clase[30];
Ahora tenemos:
clase[0]
clase[1]
clase[2]
...
clase[29]
Y cada posición tiene:
nombre
legajo
promedio

Ejemplo

```
clase[0].legajo = 1234;
clase[0].promedio = 8.5;
```

---

33. RECORRER UN ARREGLO DE STRUCTS

Utilizamos un for.

```
for(int i = 0; i < tamano; i++)
{
    printf("%s\n", clase[i].nombre);
    printf("%d\n", clase[i].legajo);
    printf("%.2f\n", clase[i].promedio);
}
```

La idea es:

clase[0]

clase[1]

clase[2]

clase[3]

...

Y en cada una accedemos a sus campos.

---

34. FUNCIONES CON ARREGLOS DE STRUCTS

También podemos enviar un arreglo de Structs a una función.

Por ejemplo:

```
float mejorPromedio(struct Alumno clase[], int tamano)
{
    float mayor = clase[0].promedio;

    for(int i = 1; i < tamano; i++)
    {
        if(clase[i].promedio > mayor)
        {
            mayor = clase[i].promedio;
        }
    }

    return mayor;
}
```

Uso:

```
float mejor = mejorPromedio(clase, 30);
```

---

35. STRUCTS CON ARREGLOS INTERNOS

Esto también aparece explícitamente en el PDF.

Una Struct puede tener un arreglo adentro.

Ejemplo:

```
struct Estudiante
{
   char nombre[50];
   float notas[3];
};
```

Ahora tenemos:

```
Estudiante
├── nombre
└── notas
   ├── notas[0]
   ├── notas[1]
   └── notas[2]
```

Acceder

estudiante.notas[0]

significa:

Primera nota del estudiante.

Calcular promedio

```
float calcularPromedio(struct Estudiante e)
{
   float suma = 0;

   for(int i = 0; i < 3; i++)
   {
      suma += e.notas[i];
   }

   return suma / 3;
}
```

Acá estamos combinando:

- Struct
- arreglo
- for
- acumulador
- función

- return

---

36. STRUCTS ANIDADOS

Una Struct puede contener otra Struct.

Por ejemplo:

```
struct Fecha
{
   int dia;
   int mes;
   int anio;
};
```

Y:

```
struct Persona
{
   char nombre[50];
   struct Fecha nacimiento;
};
```

Ahora:

```
Persona
├── nombre
└── nacimiento
    ├── dia
    ├── mes
    └── anio
```

Acceder

persona.nacimiento.dia

persona.nacimiento.mes

persona.nacimiento.anio

Es simplemente ir entrando:

```
persona
 ↓
nacimiento
 ↓
dia
```

---

37. COMBINAR STRUCTS + PUNTEROS

Acá empezamos a llegar a ejercicios bastante más avanzados.

Por ejemplo:

```
void aumentarSalario(struct Empleado *emp, float porcentaje)
```

```
{
    emp->salario += emp->salario * porcentaje / 100;
}
```

Tenemos:

struct

\+

puntero

\+

función

\+

modificación

La llamada:

```
aumentarSalario(&empleado, 10);
```

significa:

"Te paso la dirección de este empleado para que puedas modificarlo."

Y dentro:

```
emp->salario
```

permite acceder a su salario.

---

38. ARREGLOS DE STRUCTS + PUNTEROS

También podemos modificar directamente elementos de un arreglo de Structs.

Por ejemplo:

```
void actualizarKilometraje(struct Vehiculo flota[], int tamano)
{
    for(int i = 0; i < tamano; i++)
    {
        if(flota[i].anio < 2015)
        {
            flota[i].kilometraje += 1000;
        }
    }
}
```

La función recorre:

```
flota[0]
flota[1]
flota[2]
...
```

Y modifica directamente los datos.

---

39. EJEMPLO COMPLETO: TRANSFERENCIA BANCARIA

Este es prácticamente el ejercicio más integrador del primer PDF.
Tenemos:

```
struct CuentaBancaria
{
    int numero;
    float saldo;
    int estado;
};
```

Donde:
estado = 1 → activa
estado = 0 → inactiva
La función recibe:
struct CuentaBancaria *origen
struct CuentaBancaria *destino
porque necesitamos **modificar los saldos originales**.
Conceptualmente:
Cuenta origen
saldo = $10000
↓
transferir $3000
↓
saldo = $7000

Cuenta destino
saldo = $5000
↓
recibe $3000
↓
saldo = $8000

¿Qué debemos comprobar?
Primero:
¿Origen activa?
Después:
¿Destino activa?
Después:
¿Origen tiene suficiente saldo?
Si todo está bien:
origen.saldo -= monto

destino.saldo += monto
Y:
return 1;
Si algo falla:
return 0;
Y **no modificamos nada**.
Este ejercicio junta:

- Structs
- Punteros
- ->
- if
- operadores lógicos
- comparación
- modificación
- return

---

40. RESUMEN DE SÍMBOLOS

Símbolo	**Significado**
&x	Dirección de x
*p	Valor al que apunta p
int *p	p es puntero a int
x[i]	Elemento i de un arreglo
.	Acceder a campo de una Struct
->	Acceder a campo mediante puntero a Struct
'\0'	Fin de un String
[]	Arreglo
return	Devuelve un resultado